Neural Networks and Deep Learning
23CSE473

Name – M. Thanuhya
Roll no – CH.SC.U4CSE23033
CSE – A

Exercise 5: Effect of Regularization on Neural Networks

Objective

Investigate overfitting and techniques to improve generalization.

Task 31: Train the Same MLP with Different Regularization Techniques

The same Multi-Layer Perceptron (MLP) was trained on the Fashion-MNIST dataset using four different regularization settings:

- Model A: No Regularization
- Model B: Dropout (0.5)
- Model C: L2 Weight Decay
- Model D: Early Stopping

The optimizer, network architecture, batch size, learning rate, and dataset were kept unchanged to ensure a fair comparison.

Python Code

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import time

from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Dropout
from tensorflow.keras.regularizers import l2
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.optimizers import Adam
```

```python
# Load Fashion-MNIST
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()

# Normalize
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0

print(X_train.shape)
print(X_test.shape)
```

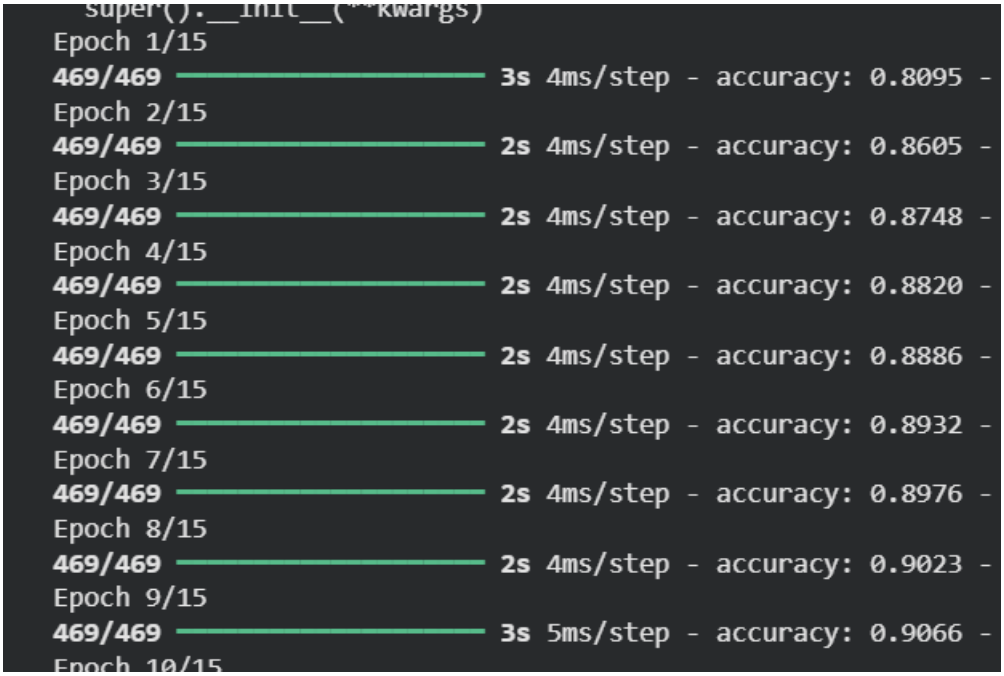
```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-d
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-d
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-d
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-d
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
(60000, 28, 28)
(10000, 28, 28)
```

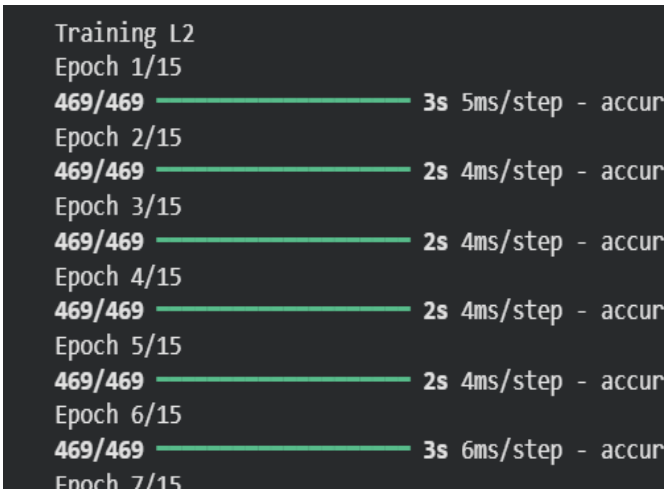
```python
histories = {}
results = {}

def build_model(mode):

    model = Sequential()

    model.add(Flatten(input_shape=(28,28)))

    if mode=="dropout":
        model.add(Dense(128,activation='relu'))
        model.add(Dropout(0.5))

    elif mode=="l2":
        model.add(Dense(
            128,
            activation='relu',
            kernel_regularizer=l2(0.001)
        ))

    else:
        model.add(Dense(128,activation='relu'))

    model.add(Dense(64,activation='relu'))
    model.add(Dense(10,activation='softmax'))

    model.compile(
```

```
        optimizer=Adam(),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    return model

models = {
    "No Regularization":"none",
    "Dropout":"dropout",
    "L2":"l2",
    "Early Stopping":"early"
}

for name,mode in models.items():

    print(f"\nTraining {name}")

    model = build_model(mode)

    callbacks=[]

    if mode=="early":
        callbacks=[
            EarlyStopping(
                monitor='val_loss',
                patience=3,
```

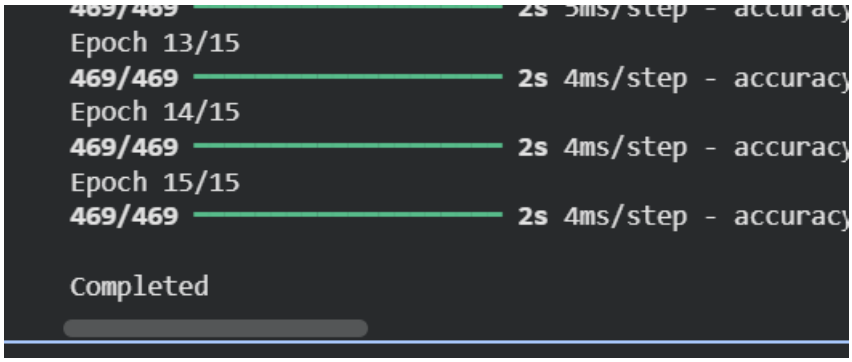
```
        history=model.fit(
            X_train,
            y_train,
            epochs=15,
            batch_size=128,
            validation_data=(X_test,y_test),
            callbacks=callbacks,
            verbose=1
        )

        end=time.time()

        loss,acc=model.evaluate(
            X_test,
            y_test,
            verbose=0
        )

        histories[name]=history

        results[name]={
            "Training Loss":history.history['loss'][-1],
            "Validation Loss":history.history['val_loss'][-1],
            "Accuracy":acc*100,
            "Training Time":end-start
        }
```

OUTPUT:


super().__init__(**kwargs)
Epoch 1/15
469/469 3s 4ms/step - accuracy: 0.8095 -
Epoch 2/15
469/469 2s 4ms/step - accuracy: 0.8605 -
Epoch 3/15
469/469 2s 4ms/step - accuracy: 0.8748 -
Epoch 4/15
469/469 2s 4ms/step - accuracy: 0.8820 -
Epoch 5/15
469/469 2s 4ms/step - accuracy: 0.8886 -
Epoch 6/15
469/469 2s 4ms/step - accuracy: 0.8932 -
Epoch 7/15
469/469 2s 4ms/step - accuracy: 0.8976 -
Epoch 8/15
469/469 2s 4ms/step - accuracy: 0.9023 -
Epoch 9/15
469/469 3s 5ms/step - accuracy: 0.9066 -
Epoch 10/15


Training L2
Epoch 1/15
469/469 3s 5ms/step - accur
Epoch 2/15
469/469 2s 4ms/step - accur
Epoch 3/15
469/469 2s 4ms/step - accur
Epoch 4/15
469/469 2s 4ms/step - accur
Epoch 5/15
469/469 2s 4ms/step - accur
Epoch 6/15
469/469 3s 6ms/step - accur
Epoch 7/15


Training Early Stopping
Epoch 1/15
469/469 3s 5ms/step - accuracy:
Epoch 2/15
469/469 2s 4ms/step - accuracy:
Epoch 3/15
469/469 2s 4ms/step - accuracy:
Epoch 4/15


Epoch 13/15
469/469 2s 4ms/step - accuracy
Epoch 14/15
469/469 2s 4ms/step - accuracy
Epoch 15/15
469/469 2s 4ms/step - accuracy
Completed

Analysis

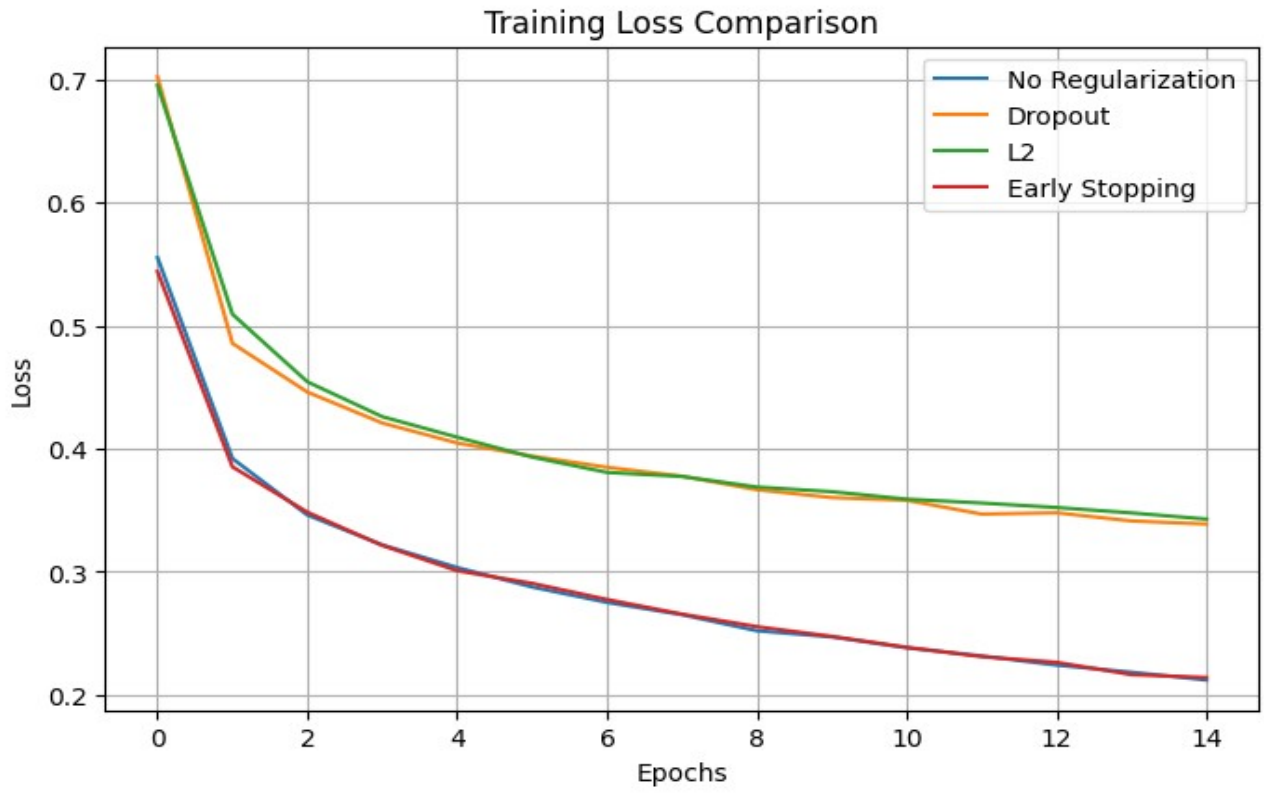
Four versions of the same MLP were trained using different regularization techniques. Keeping the remaining training parameters unchanged allowed the effect of each regularization method on model performance to be evaluated fairly.

Task 32: *Plot Training and Validation Loss*

Python code: Training Loss Comparison

```
plt.figure(figsize=(8,5))

for name, history in histories.items():
  plt.plot(history.history['loss'], label=name)

plt.title("Training Loss Comparison")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.show()
```

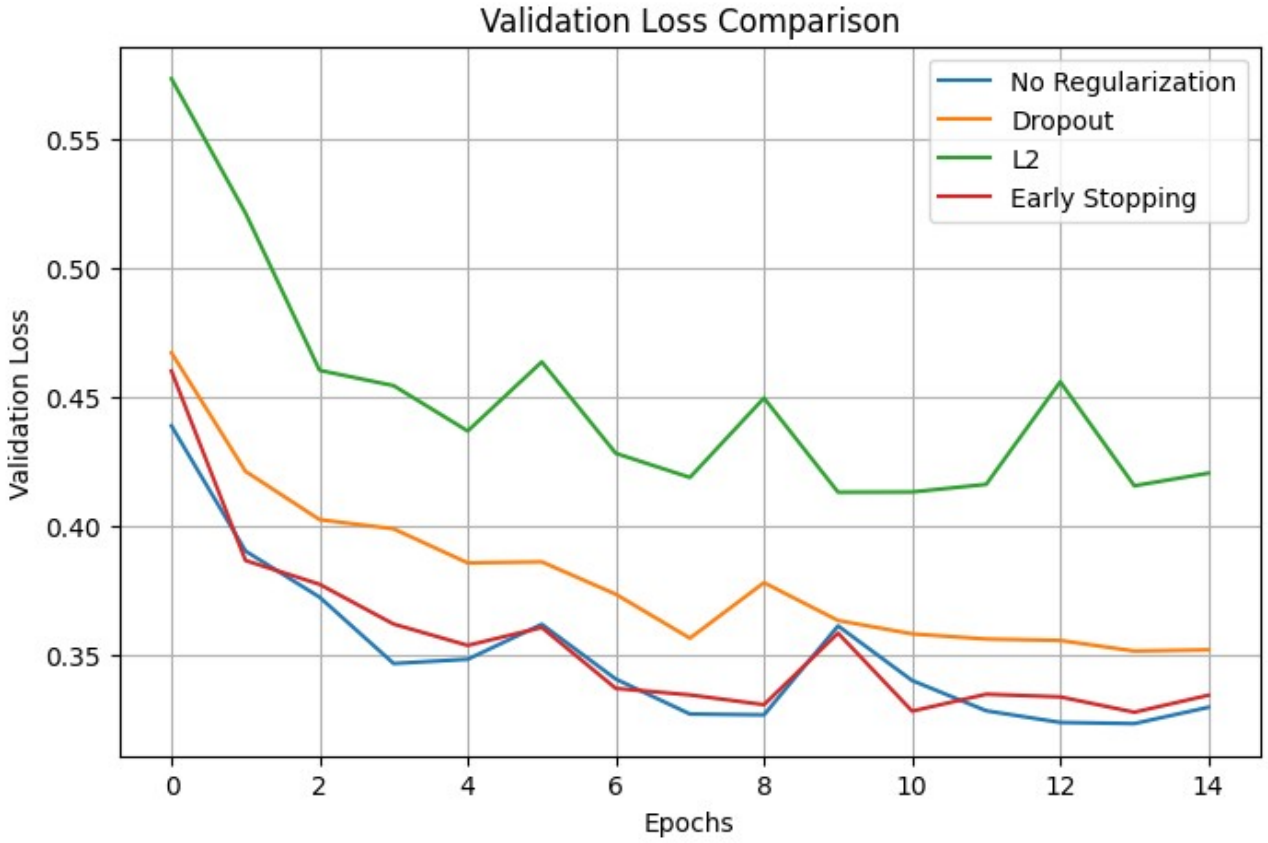
OUTPUT:



Python code: Validation Loss Comparison

```
plt.figure(figsize=(8,5))

for name, history in histories.items():
  plt.plot(history.history['val_loss'], label=name)

plt.title("Validation Loss Comparison")
plt.xlabel("Epochs")
plt.ylabel("Validation Loss")
plt.legend()
plt.grid(True)

plt.show()
```

OUTPUT:



Analysis

The training loss decreased for all four models as training progressed. The validation loss curves indicate differences in generalization performance. Early Stopping and the model without regularization achieved lower validation loss, while the L2 model maintained a comparatively higher validation loss throughout training.

Task 33: *Report Training and Validation Accuracy*

Performance Comparison Table

CODE:

```
comparison = pd.DataFrame(results).T
```

```
comparison["Training Loss"] = comparison["Training Loss"].round(4)
comparison["Validation Loss"] = comparison["Validation Loss"].round(4)
comparison["Accuracy"] = comparison["Accuracy"].round(2)
comparison["Training Time"] = comparison["Training Time"].round(2)
```

OUTPUT:

index	Training Loss	Validation Loss	Accuracy	Training Time
No Regularization	0.2122	0.3299	88.37	29.11
Dropout	0.3389	0.3521	87.23	30.71
L2	0.3429	0.4206	86.65	33.1
Early Stopping	0.2142	0.3345	88.84	31.37

Analysis

Early Stopping achieved the highest validation accuracy (88.84%), followed closely by the model without regularization (88.37%). Dropout and L2 Weight Decay also produced satisfactory results but achieved slightly lower validation accuracy in this experiment.

Task 34: Identify Signs of Overfitting

Overfitting occurs when a model performs well on the training data but fails to generalize to unseen validation data. Common signs include a continuous decrease in training loss while the validation loss remains unchanged or begins to increase. A large gap between training and validation performance also indicates overfitting.

Analysis

The training and validation loss curves show only a small difference for most models, indicating limited overfitting. Early Stopping helped prevent unnecessary training, while Dropout and L2 Weight Decay also contributed to controlling model complexity.

Task 35: Explain How Each Regularization Technique Mitigates Overfitting

- **No Regularization:** The model is trained without any mechanism to prevent overfitting, allowing it to learn all patterns present in the training data.
- **Dropout:** Randomly deactivates a fraction of neurons during training, reducing dependency on individual neurons and improving generalization.
- **L2 Weight Decay:** Adds a penalty to large weight values, encouraging the network to learn simpler models with better generalization.
- **Early Stopping:** Monitors the validation loss during training and stops training when no further improvement is observed, preventing the model from overfitting.

Analysis

Each regularization technique reduces overfitting using a different approach. Dropout reduces co-dependency among neurons, L2 limits large weights, and Early Stopping prevents excessive training. These methods improve the model's ability to generalize to unseen data.

Task 36: Recommend the Most Effective Method

Based on the experimental results, **Early Stopping** was the most effective regularization technique. It achieved the highest validation accuracy while maintaining a low validation loss and preventing unnecessary training beyond the point of optimal performance.

Analysis

Early Stopping provided the best balance between training performance and generalization. It reduced the risk of overfitting without increasing model complexity, making it the preferred regularization technique for this experiment.

Discussion

The experiment investigated the effect of different regularization techniques on the performance of a Multi-Layer Perceptron trained on the Fashion-MNIST dataset. Four training strategies were evaluated: no regularization, Dropout, L2 Weight Decay, and Early Stopping. All other training parameters were kept constant to ensure that the observed differences were due only to the regularization method.

The model without regularization achieved good training performance but is generally more susceptible to overfitting because it does not include any mechanism to control model complexity. Dropout improved generalization by randomly disabling neurons during training, forcing the network to learn more robust feature representations. L2 Weight Decay reduced the magnitude of the network weights by adding a penalty term to the loss function, encouraging simpler models. Early Stopping monitored the validation loss and automatically terminated training when additional epochs no longer improved performance.

The experimental results showed that Early Stopping achieved the highest validation accuracy (88.84%), followed closely by the model without regularization (88.37%). Dropout and L2 Weight Decay also performed satisfactorily, although their validation accuracies were slightly lower. The training and validation loss curves demonstrated that all models learned successfully, while Early Stopping effectively limited unnecessary training and reduced the possibility of overfitting.

Overall, the experiment demonstrates that regularization techniques play an important role in improving the generalization ability of neural networks. Among the evaluated methods, Early Stopping provided the best balance between performance and prevention of overfitting.

Conclusion

This experiment compared four regularization techniques for improving neural network generalization. The results showed that all methods reduced the risk of overfitting to varying degrees. Among them, Early Stopping achieved the best overall validation accuracy while maintaining stable validation loss. Therefore, Early Stopping is recommended as an effective and practical regularization technique for improving neural network performance.